

○ FHE BUIDL MUMBAI • ANALYTICS TRACK • 2026

# fheENV

Your `.env`, encrypted. Not even us.

## Fully Homomorphic Encryption

**(f)** Fhenix CoFHE

## N Next.js + wagmi

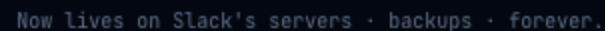
## CLI + Web + Contract

# You've done this. Today.

That message lives on someone else's server — forever.

of breaches start with a stolen credential

credential breaches in 2025 alone



## Same problem. Different server.



## Doppler

Centralized server holds your keys in plaintext.  
\$20M raised. Still reads your secrets.

⚠ Doppler can read your keys



## HashiCorp Vault

Admin has full access. \$6.4B IBM acquisition.  
Complex self-host.

⚠ Admin has full plaintext access



## AWS Secrets

US servers. \$0.40/secret/month. Subject to government subpoena.

- ⚠ US gov can subpoena it

**All three require you to trust a company — that trust is your attack surface.**

# Mathematically.

© ARCHITECTURE

## AES + FHE Envelope

## 01 Generate AES-256 key

Random per rotation. Never stored anywhere.

## 02 AES-GCM encrypt → IPFS

Blob is gibberish without the key. CID stored on-chain.

### 03 Split AES key → 2× euint128 on-chain

FHE-encrypted. No node can read it.

84 FHE.allow() per wallet

Threshold Network decrypts only to authorized address.

WHERE DATA LIVES

| LOCATION     | DATA                    | READABLE?       |
|--------------|-------------------------|-----------------|
| On-chain     | 2× euint128 AES handles | ✗ FHE encrypted |
| IPFS         | AES-encrypted .env blob | ✗ Gibberish     |
| Fhenix nodes | FHE computation         | ✗ Encrypted     |
| Your device  | Plaintext .env          | ✓ Only you      |

One FHE decrypt per pull

Other primitives can't do this.

## X ZK Proofs

Prove "I know X" — can't selectively reveal encrypted data to different parties.

**X AES / RSA alone**

Someone must hold the decryption key — that someone is your attack surface.

- ✓ FHE (Fhenix CoFHE)

Key stored as ciphertext. `FHE.allow()` grants per-wallet decrypt. Threshold Network — no single party reconstructs.

### OPERATIONS USED

```
FHE.asEuint128() - store AES key chunk
FHE.allow(handle, addr) - grant decrypt
FHE.allowThis() - re-grant on rotation
```

```
// AES key → two FHE-encrypted uint128
evuint128 hi = FHE.asEvuint128(inHigh);
evuint128 lo = FHE.asEvuint128(inLow);

FHE.allowThis(hi); FHE.allowThis(lo);
FHE.allow(hi, msg.sender);

// Grant teammate decrypt
FHE.allow(env.aesKeyHigh, member);
FHE.allow(env.aesKeyLow, member);

// Revoke = rotate → new handles, old abandoned
updateEnvironment(id, name,
    newHi, newLo, newCid,
    expectedVersion // optimistic lock
);

// Batch grant up to 100 in one tx
batchGrantAccess(id, name, address[] members);
```

## 📦 WHAT WE BUILT

# Three components. Fully working.



### Smart Contract

Solidity 0.8.25 · Sepolia

- createProject / addOwner
- updateEnvironment + optimistic lock
- grantAccess / batchGrantAccess
- revokeAccess + mandatory rotate

Deployed · Sepolia



### Web Dashboard

Next.js 14 · wagmi · @cofhe/sdk

- Wallet auth – no email/password
- Push: AES encrypt → IPFS
- Pull: FHE decrypt → view secrets
- Team: grant / revoke + audit log

Live App



### CLI Tool

Node.js · npm installable

- fheenv push / pull / run
- fheenv rotate (new AES + re-grant)
- fheenv team add / remove
- FHEENV\_PRIVATE\_KEY for CI/CD

npm i -g fheenv

## End-to-end. Working.

- 1 Push (Wallet A)**  
Paste .env → encrypt + upload → FHE-encrypt key halves → tx submitted.
- 2 Inspect Etherscan**  
**Zero readable secrets** — only FHE-encrypted bytes visible.
- 3 Grant (Wallet A → Wallet B)**  
FHE.allow() for both key halves. One transaction.
- 4 Pull (Wallet B)**  
Threshold decrypts AES key locally → secrets appear. ✓
- 5 Revoke + Rotate**  
New AES key → new FHE handles → Wallet B locked out cryptographically.

```
~/app → fheenv push --env production
  Encrypting + uploading to IPFS...
  FHE-encrypting AES key...
  ✓ Pushed (CID: Qm4xF9...)

~/app → fheenv team add 0xB2c...3f4
  ✓ Access granted

~/app → fheenv team remove 0xB2c...3f4
  Revoking + rotating AES key...
  ✓ Rotation complete. Member locked out.

~/app → FHEENV_PRIVATE_KEY=0x... fheenv run -- node app.js
  ✓ Secrets injected. No disk write.

# Etherscan calldata:
0xa3f219c8...c7b2d9f0e4a1...

# Zero plaintext ✓
```



built for **this track.**

| TRACK CRITERIA                   | FHEENV                                                                          |
|----------------------------------|---------------------------------------------------------------------------------|
| Encrypted information operations | AES-256 + FHE — never plaintext on any server                                   |
| Autonomous systems               | CI/CD pulls secrets autonomously via <code>FHEENV_PRIVATE_KEY</code>            |
| Security & user privacy          | Cryptographic access control + on-chain audit log                               |
| Fhenix FHE use case              | <code>evint128</code> , <code>FHE.allow()</code> , rotation-enforced revocation |

theENV is the **secrets layer for the encrypted web** — foundational infrastructure for any app that handles credentials without a centralized trust point.

 MARKET

**\$2.3B market. Zero FHE players.**

**\$2.3B**

Market size 2025

**\$8.9B**

Projected 2030

**\$6.4B**

HashiCorp → IBM

## THE GAP

Doppler · Vault · AWS — all readable by operator

theENV — mathematically private. Free.

## INDIA OPPORTUNITY

5M+

Software developers — fastest-growing dev market

## DPDP Act 2023

## Startup Fit

No subscription. No vendor trust. No lock-in. Free infra.



# this track.

Contract on Sepolia. Web app live. CLI on npm. IPFS via Pinata.

**fheENV**

The Doppler alternative where **not even we** can see your secrets.

© Fhenix CoFHE



- Node.js CLI

10

9

[github.com/Team-Managed/fheENV](https://github.com/Team-Managed/fheENV)

[fheenv.vercel.app](https://fheenv.vercel.app)

"Every dev in this room has a secret on Slack right now. **fheENV** means that never has to happen again."